PROJECT GUIDE · PART 1

StreetCare AI Architecture & Workflow

How a street-defect review platform for Wrenford Borough Council is built: from messy report files to clean data, three models, a searchable policy library, a ten-node agent workflow that stops for a human inspector, a secured REST service and a role-based dashboard. Explained in simple words, with a diagram for every flow.

Python · pandas & Decimal · scikit-learn · TF-IDF · ChromaDB RAG · Gemini LLM · LangGraph + checkpoint · FastAPI · SQLAlchemy · Streamlit

---

Covers all 6 entry points · 11 packages · 10-node review graph · 3 roles
Prepared for the StreetCare AI team · built from the project specification

Contents

1 The project in one page

2 The big picture: architecture

3 How the six entry points stack up

4 Folder structure

5 Flow 1: Data cleaning

6 Flow 2: Model training

7 Flow 3: Knowledge base (RAG indexing)

8 Flow 4: Report screening

9 Flow 5: Evidence & predictions

10 Flow 6: The ten-node review workflow

11 Flow 7: The human decision gate

12 Flow 8: The secured REST service

13 Flow 9: The street care chat

14 Flow 10: The dashboard

15 One review, end to end

16 Files: who writes, who reads

17 Design choices worth explaining

18 How to run it

STATUS OF THE CODE

The repository ships `config/config.py`, the data and the 16 PDFs; every other module is still empty. This guide describes the system exactly as the specification defines it. Row counts, document counts and every threshold are the specification's own figures; model scores are printed by `run_model_training.py` once it is built, so they are shown as minimums, not invented.

1. The project in one page

Wrenford Borough Council looks after **48 streets** in **4 districts** (Town Centre, North, South, Villages). Residents, businesses, councillors, bus operators and schools report street defects (potholes, broken paving, blocked drains, low branches, faulty street lights) through the website, the StreetCare app or the contact centre. Every report lands in one queue in the Street Care Office, and nothing is repaired until a **Highways Inspector** appointed for the district decides it.

Problem today	What the platform does
Every channel writes a report its own way: `SC-50118`, `4.5 cm`, `210 min`	Reads every answer as the Report Rules read it: one reference format, mm / m / degrees / hours, exact decimals
Carriageway and footway standards look alike but set different limits	Reads each requirement and its figure from the report's **own** standard PDF at run time; nothing is typed into code
Attached notes sometimes give a different measurement than the report	Reads every sentence, labels it with an NLP model, and flags figures that disagree or fill a blank
Street works, busy roads and tall columns need coordinating first	Counts live permits on the street and checks the three coordination triggers
No idea which defects will come back or how long repairs take	Two models estimate repeat likelihood and expected crew hours, to inform, never to decide
Customer services must not leak predictions or identify households	A chat that answers from council documents only; code, not the model, answers about reports and streets, by role

40
open reports in the live queue (from 45 rows)

16 / 136
council PDFs / numbered sections indexed

3 models
repeat likelihood · crew hours · sentence type

10 nodes
in the LangGraph review workflow

7 / 6 / 5
LLM calls for ORDER / INSPECT / REFER

3 roles
16 seed accounts · 2 inspectors per district

Tech stack at a glance

Layer	Tools
Language & core	Python 3, `decimal.Decimal` (round half up), regular expressions, TypedDict
Data	pandas 3 (text-first reading, nullable `Int64`), CSV + JSON outputs
Machine learning	scikit-learn: Logistic Regression, Random Forest Regressor, TF-IDF + Logistic Regression; joblib
Retrieval (RAG)	pypdf, sentence-transformers `all-MiniLM-L6-v2` (CPU), ChromaDB (cosine)
Agents & LLM	LangGraph (StateGraph, interrupt, SQLite checkpointer), Google Gemini via `google-genai`
Storage	SQLite through SQLAlchemy ORM: 6 audit tables

Service & UI	FastAPI, Pydantic, Uvicorn (port 8081) · Streamlit dashboard (port 8080)
Security	PBKDF2-HMAC-SHA256 (200,000 rounds, 16-byte salt), random session tokens, role permissions

2. The big picture: architecture

The platform has **four layers**. Raw files at the top are turned by offline jobs into saved artifacts; the live service loads those artifacts once and serves every request from them.

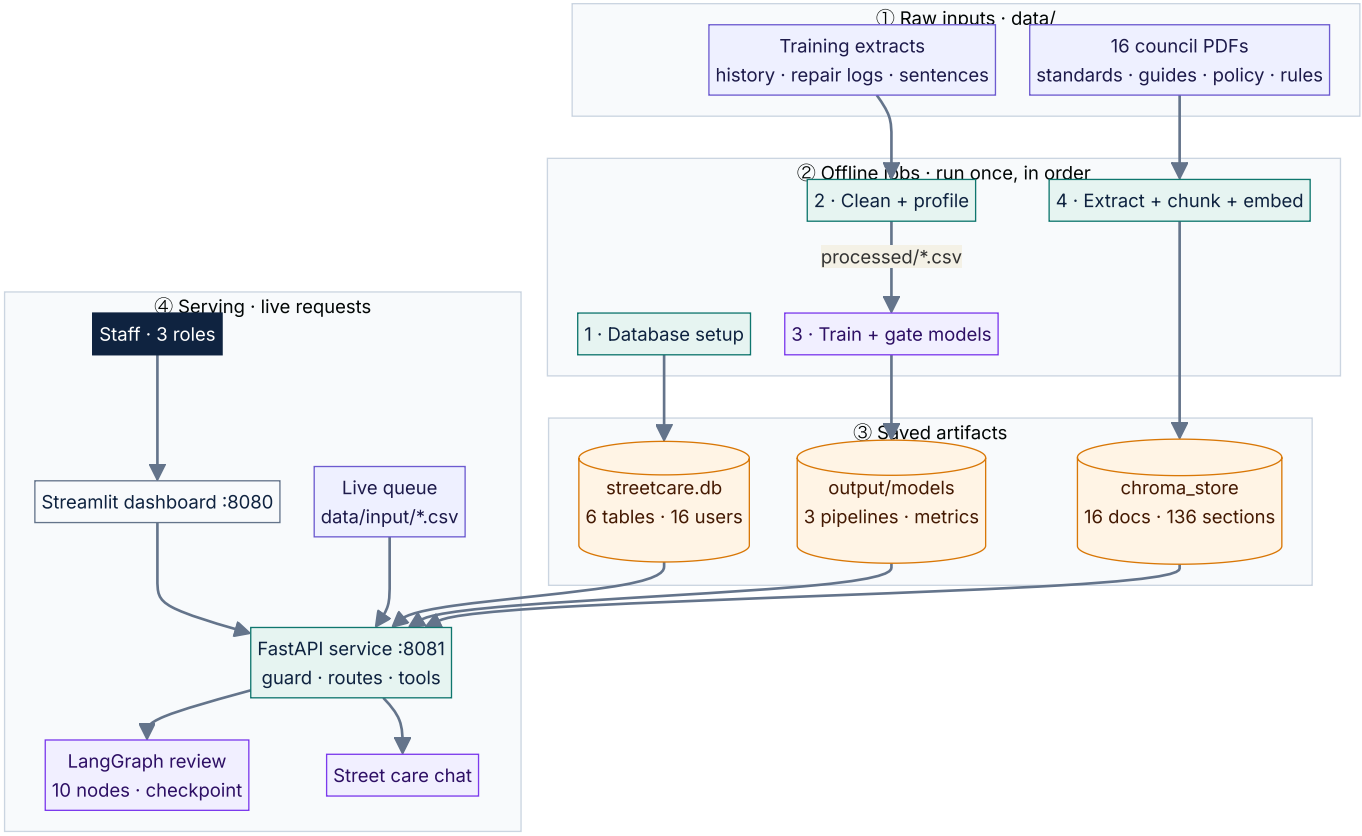


Figure 1: Four layers: inputs → offline jobs → saved artifacts → live serving

Offline (batch) side

Runs once, in order, from the terminal: create the database and accounts, clean the five training extracts, train and gate three models, index the 16 PDFs. It writes into `database/`, `processed/` and `output/`.

Online (serving) side

The FastAPI service checks every required file, loads the live queue, models and index once, works out the queue, and serves the dashboard. Reviews run as LangGraph workflows that pause for a human.

IN ONE LINE

Offline jobs **produce** artifacts → the service, the workflow and the chat **consume** them. Nothing is retrained while a request is served, and no repair is ordered without a person.

3. How the six entry points stack up

Each entry point reads what the one before it wrote, which is why the order of running matters. The service refuses to start if anything it needs is missing, and names the step that makes it.

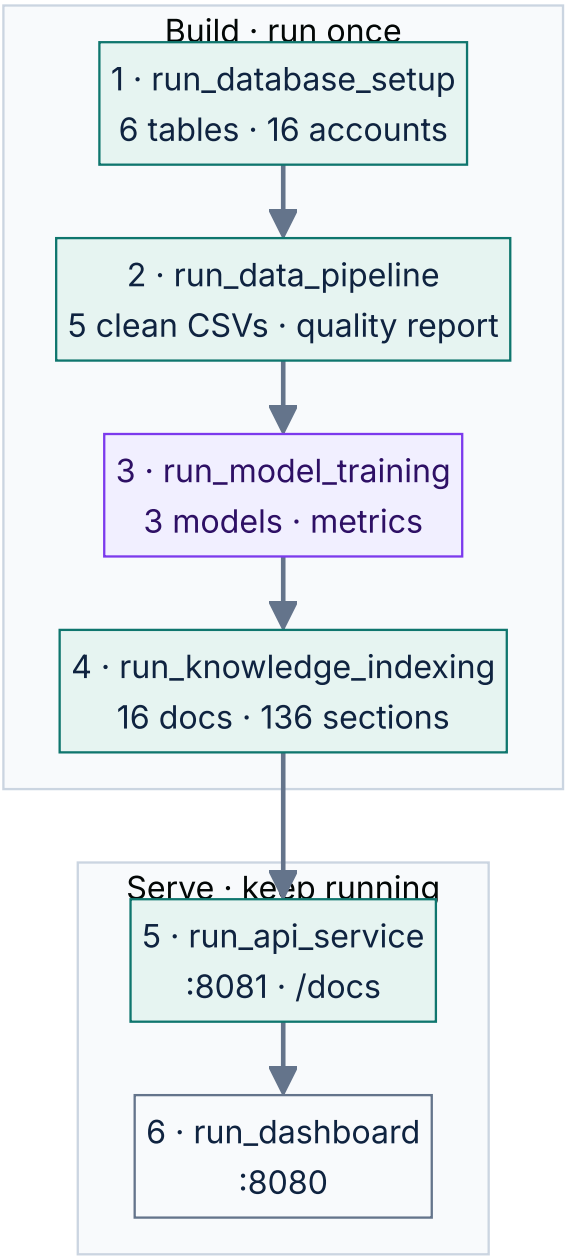
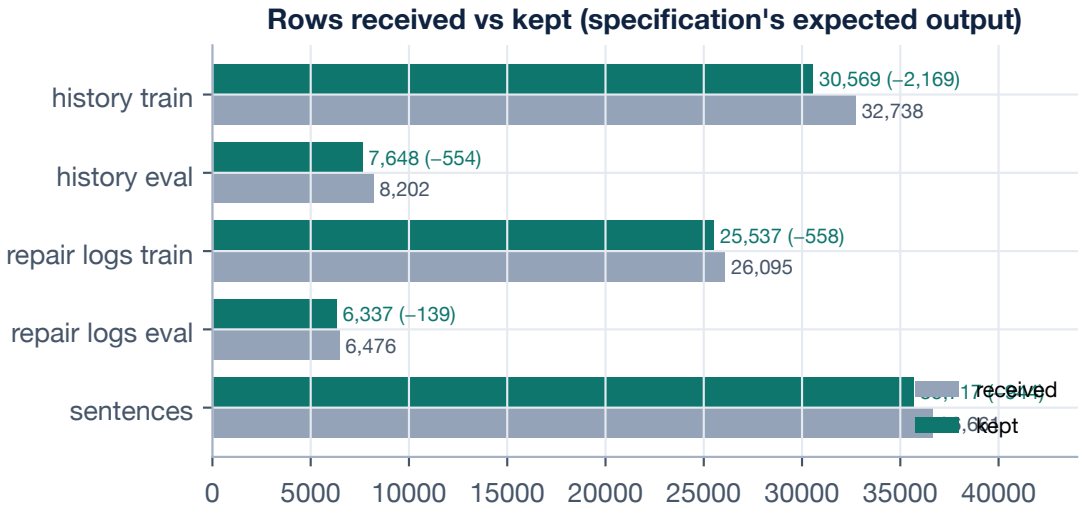


Figure 2: Six entry points, each building on the last

#	Entry point	What it builds (simple words)	Must report
1	`run_database_setup.py`	Creates the six audit tables and the 16 seed accounts with salted password hashes. Safe to run twice.	accounts created: 16 · second run: 0, already present: 16
2	`run_data_pipeline.py`	Cleans five raw extracts (references, spellings, units, duplicates, revisions), saves them, profiles before vs after.	received → kept (removed) per dataset

3	`run_model_training.py`	Adds requirement counts and trigger flags, trains three models, scores them on held-out data, refuses any below its minimum.	held-out score, train score and gap for each
4	`run_knowledge_indexing.py`	Extracts the 16 PDFs, splits sections, cuts passages, embeds them into Chroma, reads back §6 of each standard as a check.	16 documents, 136 sections, N passages
5	`run_api_service.py`	Checks required files, loads everything once, builds the workflow and the queue, serves the API.	docs URL, or each missing file
6	`run_dashboard.py`	Signs users in and shows each role only its pages, all fed by the API.	Streamlit URL on :8080



The pipeline's expected row counts, straight from the specification

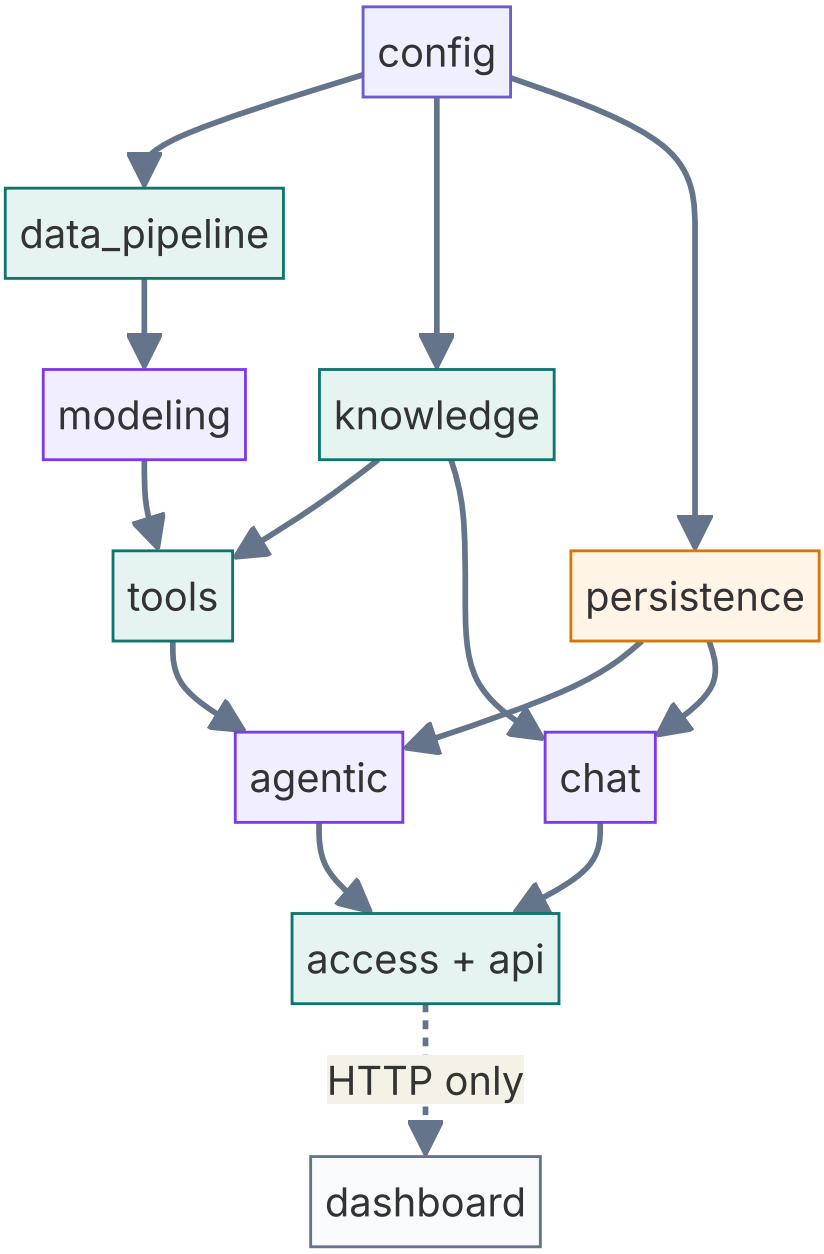
4. Folder structure

```
Project/
├── config/config.py          # every path, mapping, pattern, threshold, role and message (supplied
complete)
├── data/                     # raw inputs (read-only)
│   ├── report_history_{train,eval}.csv   # 32,738 / 8,202 rows · target reported_again
│   ├── repair_logs_{train,eval}.csv      # 26,095 / 6,476 rows · target actual_crew_hours
│   ├── statement_sentences.csv           # 36,661 rows · target sentence_type (9 types)
│   ├── input/                            # live: open_defect_reports · report_documents ·
street_works_register
│   └── docs/                             # 16 PDFs: 5 standards · 8 guides · policy · rules · casebook
├── data_pipeline/            # loader · common_cleaning · 6 cleaners · feature_builder ·
quality_profiler
├── modeling/                 # registry · evaluation · repeat · crew_hours · sentence
├── knowledge/                # pdf_extraction · chunk_splitting · vector_index · standard_reading
├── tools/                    # report_data · screening · evidence · prediction · retrieval · decision
· review_document
├── persistence/              # database · models (6 tables) · repository
├── agentic/                  # state · language_model · agents (7) · nodes (10) · graph
├── chat/                     # conversation_memory · street_care_assistant · record_answer
├── access/                   # password_security · permissions
├── api/                      # schemas · routes
├── dashboard/                # api_client · sign_in · report_queue · report_detail · decision_queue ·
audit_trail · street_care_chat
├── run_*.py                  # the six entry points
├── tests.py                  # the test suite
├── processed/                # generated: cleaned CSVs
├── output/                   # generated: data_quality_report.json · model_metrics.json · models/
├── database/                 # generated: streetcare.db · checkpoint.db · chroma_store/
└── .env                      # GEMINI_API_KEY, GEMINI_MODEL (never in code, never in git)
```

RULE OF THE PROJECT

Nothing under `data/` or `config/` is ever written to, and **no requirement or limit is written in code or config**: every figure is read from its own defect standard. Every generated file lands in `processed/`, `output/` or `database/`, so the whole project can be rebuilt from scratch.

Dependencies point one way



A lower layer never imports a higher one; the dashboard never imports project code at all

5. Flow 1: Data cleaning

Every extract arrives **deliberately messy**. Each value passes through the same shared repairs, in a fixed order. A value the rules do not list is **refused** with an error that names the column and the values: never guessed, never silently dropped.

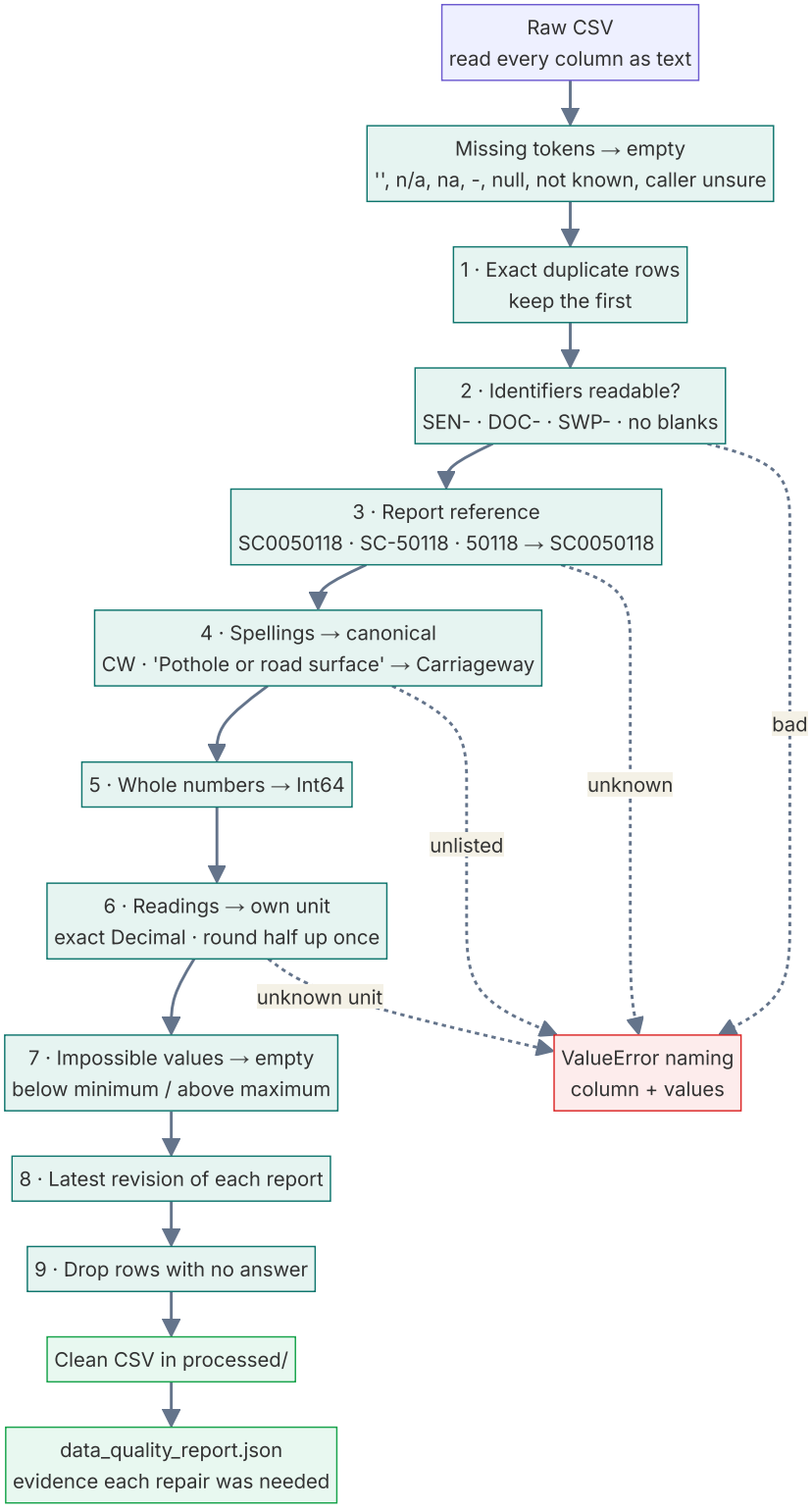


Figure 3: The shared cleaning chain (each dataset uses the steps its rules need)

Units, read the Report Rules' way

Written	Read as
4.5 cm	45 mm
1.75 in	44 mm (44.45 → half up)
4 ft	1.2 m
45 min	0.8 hours

Why exact decimals?

A limit is met or missed by the last digit. `Decimal` with one half-up rounding at the reading's own precision means 31.75 mm is 32 mm every time, on every machine, and the worked example in the Report Rules comes out exactly as written.

Why "latest revision"?

A report sent again with an answer added has a higher `revision_number`. Only the highest revision *is* the report: earlier ones are never screened and never learned from.

Why refuse rather than guess?

A guessed unit silently passes or fails the wrong report. An error that names the value is fixed once, at the source.

Dataset	Received	Kept	Removed	Answer column
report_history_train	32,738	30,569	2,169	reported_again
report_history_eval	8,202	7,648	554	reported_again
repair_logs_train	26,095	25,537	558	actual_crew_hours
repair_logs_eval	6,476	6,337	139	actual_crew_hours
statement_sentences	36,661	35,717	944	sentence_type
open_defect_reports (live)	45	40	5	none
street_works_register (live)	29	28 (23 live)	1	none

TEXT GETS ITS OWN REPAIRS

- Tags matched by `MARKUP_PATTERN` are removed; `&`-style entities become characters; every run of whitespace (no-break spaces too) becomes one space.
- A sentence written wholly in capitals is lower-cased (the only way it can be restored); its length is recounted from the clean text.
- Sentences with fewer than 3 words, or no type, are left out of training.

6. Flow 2: Model training

Three models are trained on the clean train files and scored on the held-out eval files they never saw. The repeat model learns from more than the raw answers: each past report is **screened under the standards read from the PDFs**, adding requirement counts and trigger flags. A model below its minimum stops the run and nothing is saved.

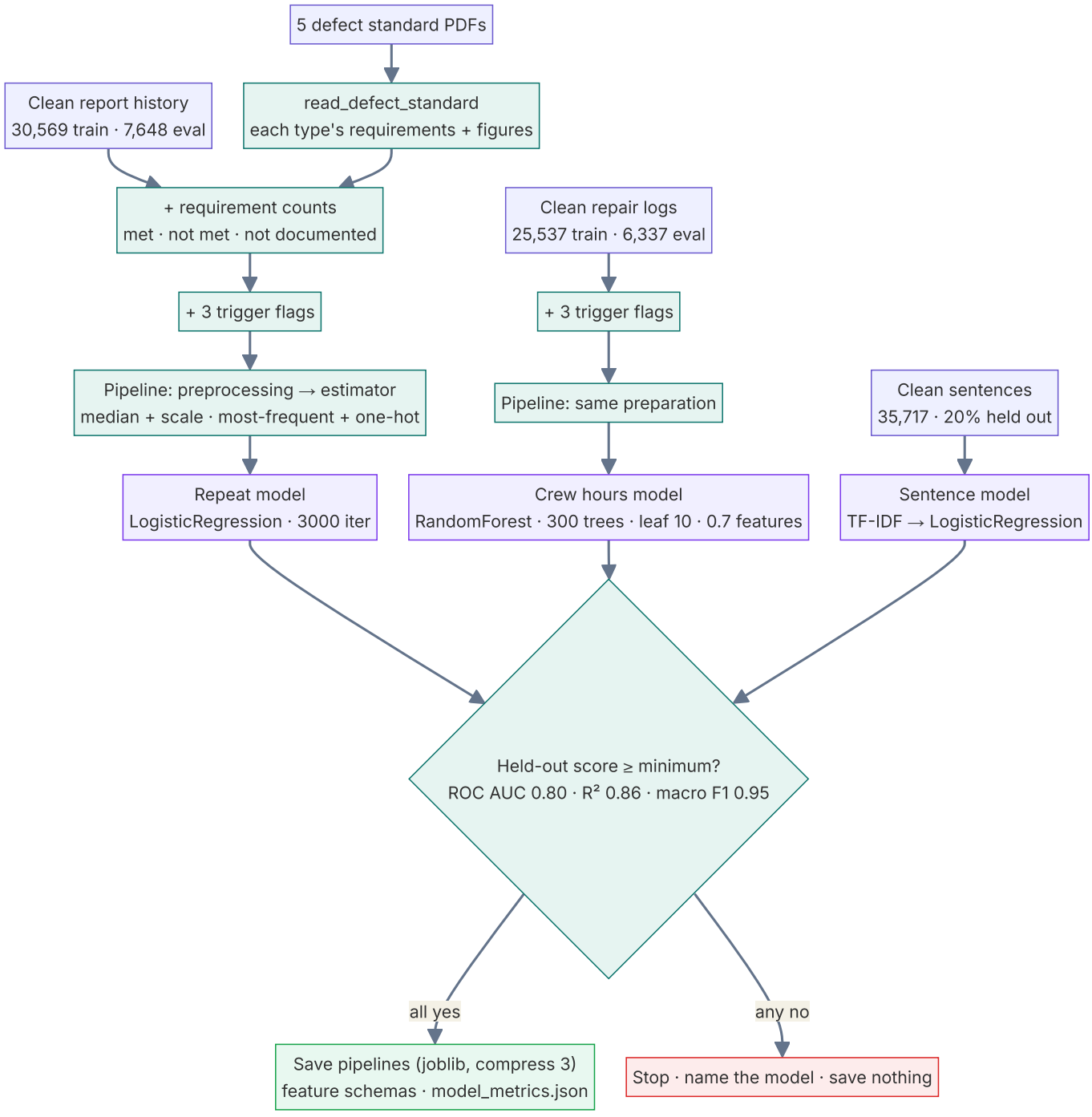


Figure 4: From clean history to three gated, saved models (RANDOM_SEED = 1313 everywhere)

Model	Predicts	Inputs	Score	Minimum	Used live as

Repeat	Will this defect be reported again after repair? (Yes = 1)	17 numbers incl. 3 requirement counts + 3 trigger flags; 10 categories	ROC AUC	0.80	likelihood to 0.01 · LOW / MEDIUM / HIGH
Crew hours	Hours the repair will really take on site	14 numbers incl. planned hours + trigger flags; 6 categories	R^2, MAE	0.86	hours to 0.1 · overrun if > 1.3 × planned
Sentence	Which of 9 types a sentence is	raw sentence text	macro F1	0.95	labels every sentence of a report's documents

BEHAVIOURAL CHECKS IN THE SPECIFICATION

A good score is not enough; each model must also **behave sensibly**:

- Repeat: turning one requirement from Met to Not met, or adding one conflicting live permit, raises the score for at least 95% of held-out reports; live likelihoods fall in all three bands.
- Crew hours: a traffic-sensitive street instead of a local road, or one more conflicting permit, lengthens the forecast for at least 90% of held-out repairs; its average miss is below 0.75 × a simple baseline's; no forecast is below the fewest hours ever logged.
- Sentence: all nine types exist and every type is predicted for some held-out sentence.

Why one pipeline per model?

Filling gaps, scaling and one-hot encoding are fitted on training rows only and saved with the estimator, so a live report with gaps is scored exactly as the model was trained, with no leakage.

Why a saved feature schema?

The exact column order the model was fitted on is written to JSON; a live report is laid out in that order before scoring, whatever order the table holds.

Output: `output/models/repeat_model.pkl`, `crew_hours_model.pkl`, `sentence_model.pkl`, `repeat_features.json`, `crew_hours_features.json` and `output/model_metrics.json` (held-out score, train score and gap for each).

7. Flow 3: Knowledge base (RAG indexing)

The 16 PDFs are the **only** source of any requirement, rule or check the system states. Indexing makes them searchable and traceable; standard reading turns each defect standard's section 4 into requirements with exact figures.

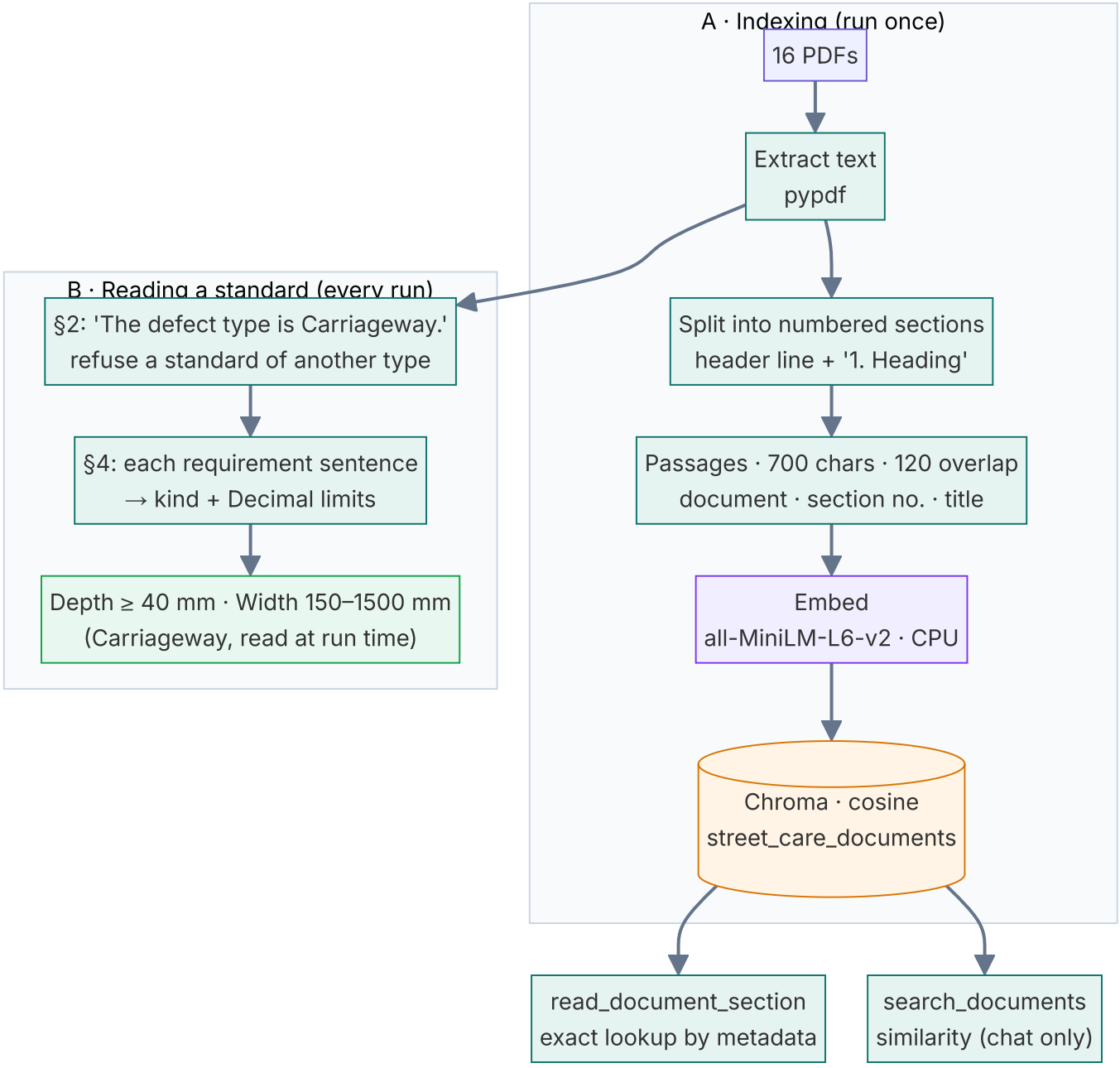


Figure 5: Index once; read each standard's rules from its own document

Documents	Sections the system reads
5 Defect Standards (Carriageway, Footway, Drainage, Vegetation, Street lighting)	§2 defect type · §4 requirements · §6 what an inspection records · §7 works order · §8 resident update

8 Statement Guides (Measurement, Location, Photograph, Hazard, Incident, Traffic and use, Previous reports, Weather and water)	§3 what the reviewer checks
Highways Policy	§6 coordination triggers · §7 the three decisions · §8 documents and the coordination brief
Report Rules	how every answer is read; §13 worked example
Past-Report-Decisions-Casebook	indexed, but **excluded from the chat**

WHY SECTIONS ARE READ BY METADATA, NOT SIMILARITY

The five standards share one template and in places the same sentences. A similarity search for "what an inspection records" could return the Footway text for a Carriageway report. Reading *document + section number* directly cannot. The indexing run prints §6 of every standard; two types showing the same text is the failure this step exists to catch.

8. Flow 4: Report screening

One open report is judged under its own standard first, then its linked type's. Every requirement ends **Met**, **Not met** or **Not documented**; the street is checked for three coordination triggers; and a fixed precedence gives the assessment and the decisions left open.

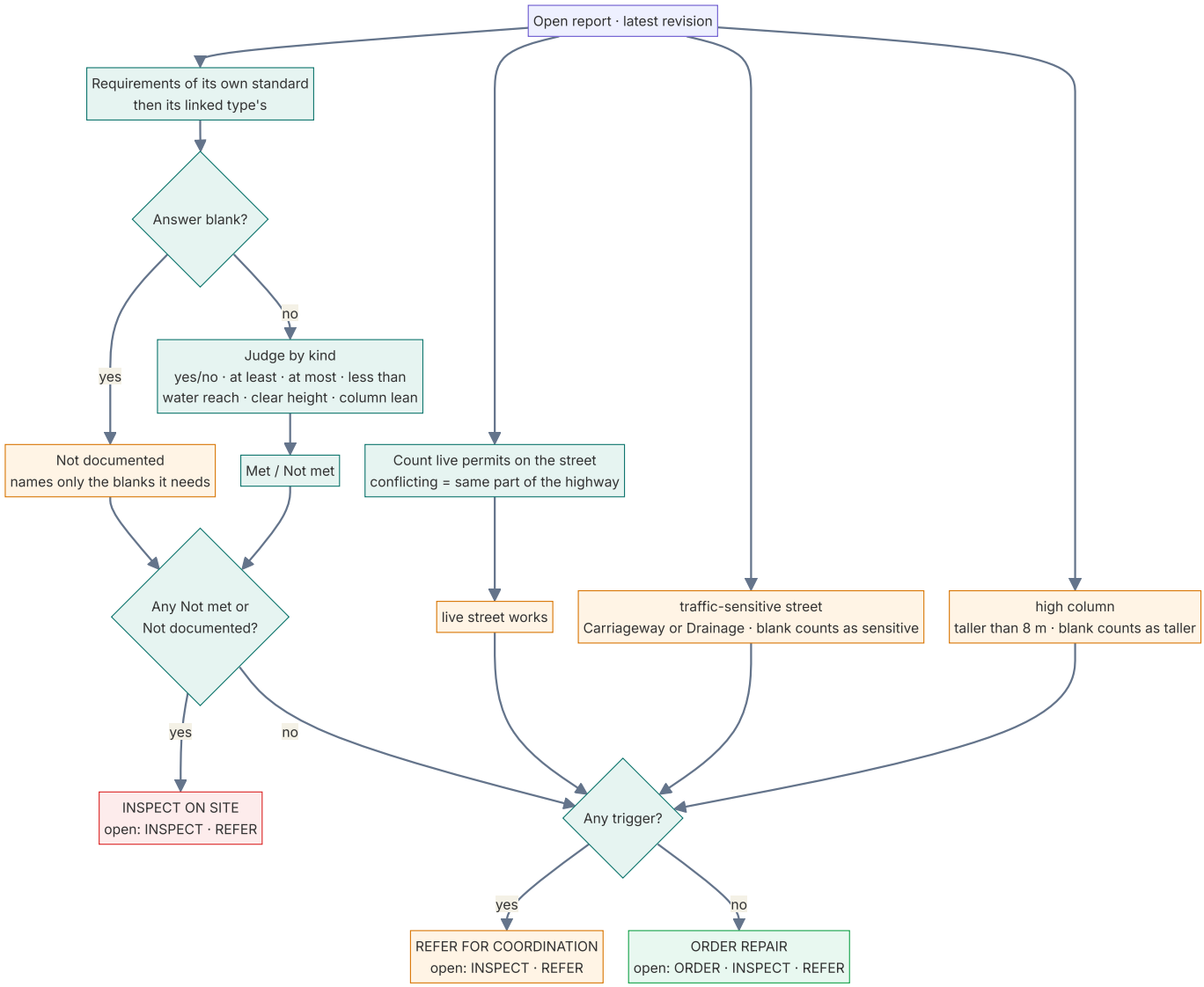


Figure 6: Requirements first, then coordination, then order

Requirement kind	Met when
Yes/no: location fixed, photograph, highway land, column number	Yes → Met · No → Not met · blank → Not documented
"at least" (depth, water depth)	value ≥ figure (exactly at the figure is Met)
"at least ... at most" (width)	minimum ≤ value ≤ maximum
"less than"	value < figure (exactly at the figure is Not met)
Water reach	water on the footway → Met · on the carriageway → Met at or beyond the figure
Clear height	footway limit over the footway · carriageway limit over the carriageway

Column lean	lamp out or day burning → Met · leaning column → held to the figure

WORKED EXAMPLE · REPORT RULES §13

A contact-centre report written `CW` linked `DR` on a street written `TS`, under a made-up standard (depth ≥ 30 mm, width 100–1000 mm, water ≥ 40 mm, reach ≥ 1.5 m on the carriageway):

Answer	Read as	Result
depth 1.25 in	31.75 → 32 mm	Met
width 3.9 in	99.06 → 99 mm	Not met
standing water 1.5 in	38.1 → 38 mm	Not met
water on ROAD, reach "caller unsure"	reach blank	Not documented (asks for the reach alone)

Met 1 · Not met 2 · Not documented 1 → **INSPECT ON SITE**. The traffic-sensitive trigger also applies, but the requirements come first.

A blank never slips through

A blank street category counts as traffic-sensitive and a blank column height counts as taller than 8 m, so missing data can only add caution.

Pure and repeatable

Same report + same standards → same screening. It is also how every past report gets its requirement counts for training.

9. Flow 5: Evidence & predictions

Two more sets of facts are worked out without any LLM: what the attached documents say, and what the models expect. They **inform** the inspector; neither changes an answer or decides anything.

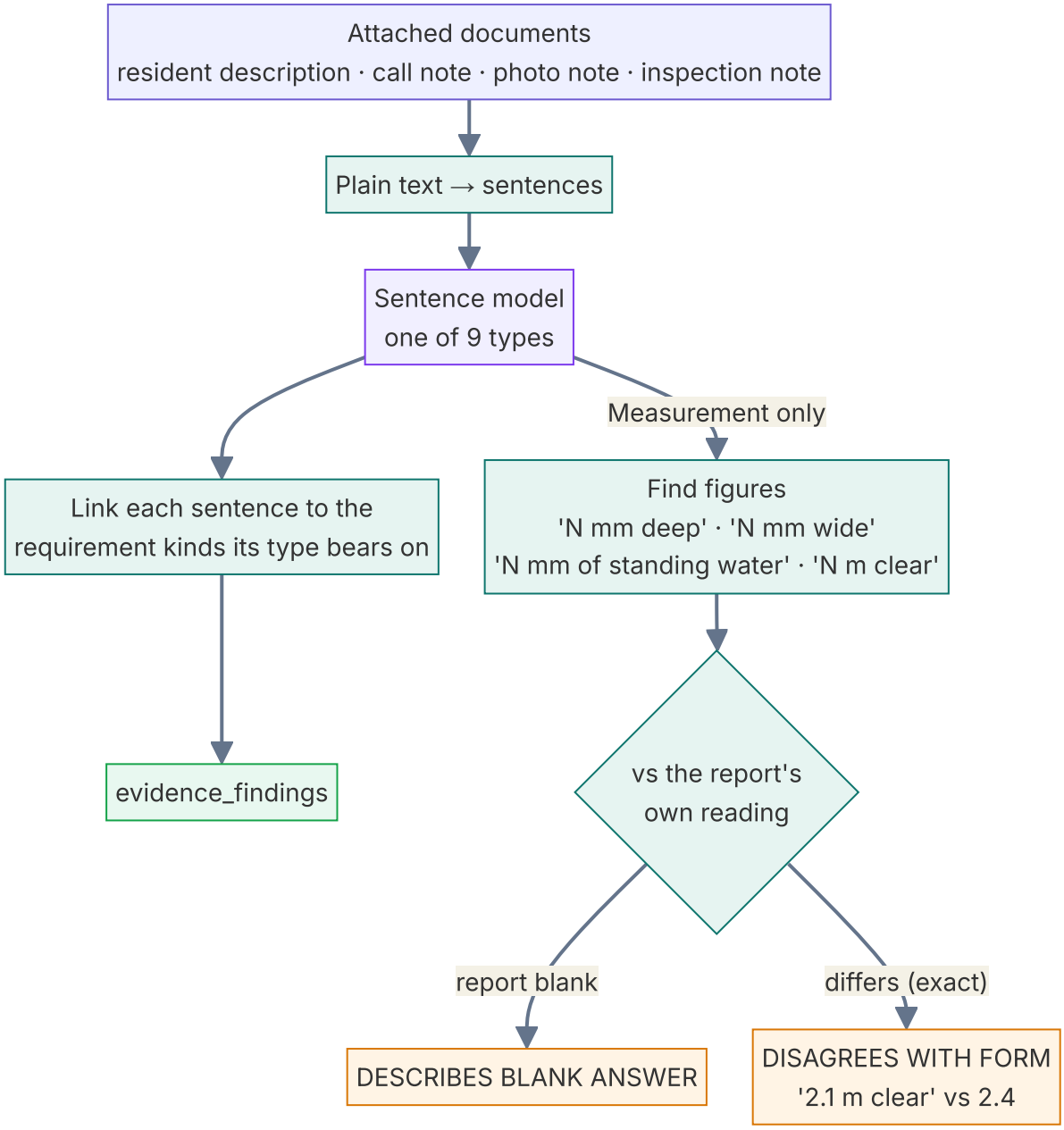


Figure 7a: Reading every sentence of a report's documents

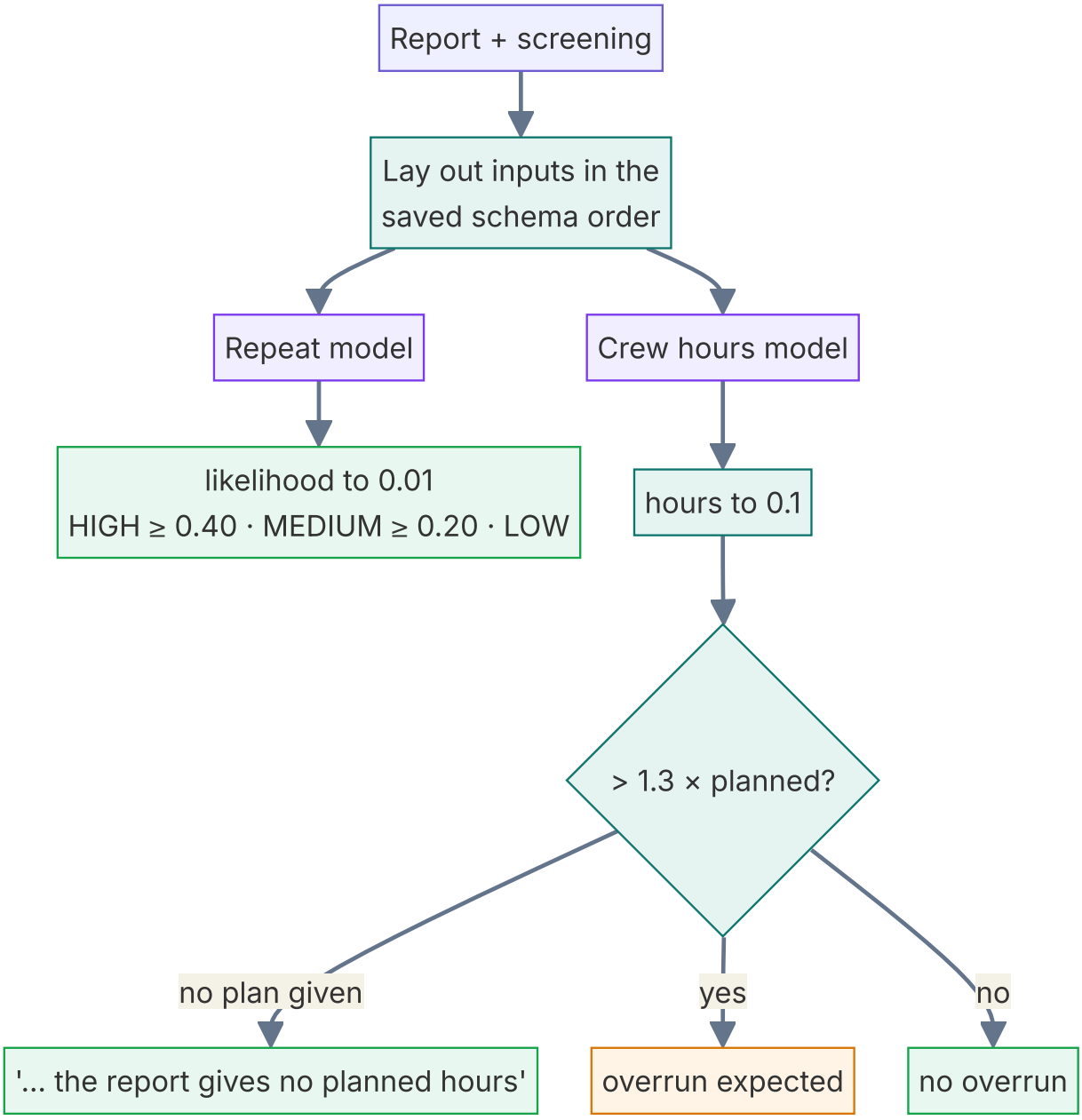


Figure 7b: The two predictions and how they are worded

WHERE PREDICTIONS MAY NEVER GO

They never order the queue, never set the assessment, never appear in a works order, inspection request or resident update (the words *likelihood*, *probability* and *expected crew hours* are forbidden there), and a Customer Services Adviser never hears one.

10. Flow 6: The ten-node review workflow

A review is a **LangGraph state machine**. Node 1 gathers every fact with tools; five agents (one Gemini call each) interpret them; the run **pauses** for a Highways Inspector; then up to two writers produce what the decision calls for. There are exactly two conditional branches, both chosen by `route_report_decision`.

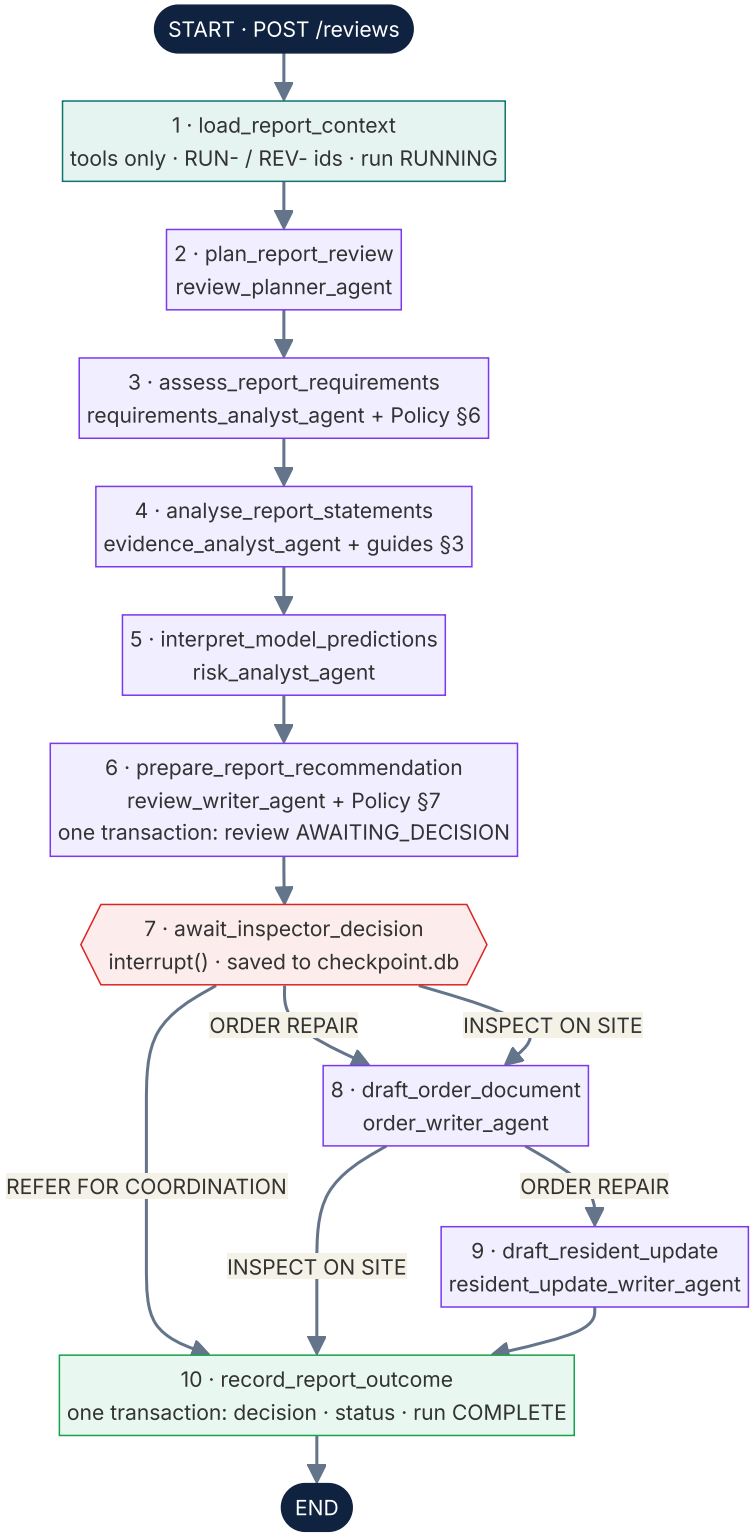


Figure 8: Ten nodes, the human gate and the two branches; purple nodes cost one LLM call each

The seven agents

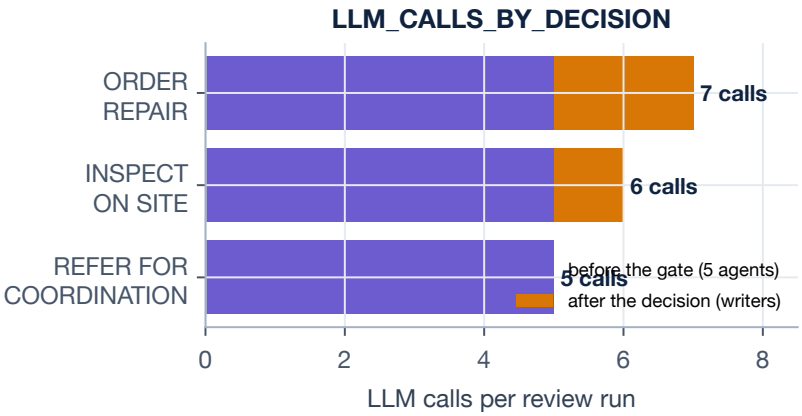
Agent	Returns
planner	PRIORITY · CHECKS · RATIONALE
requirements analyst	MET · NOT MET · NOT DOCUMENTED · TRIGGERS · SUMMARY
evidence analyst	EVIDENCE FOUND · DISAGREEMENTS · CHECKS TO MAKE · SUMMARY
risk analyst	LIKELIHOOD · EXPECTED CREW HOURS · SUMMARY
review writer	RECOMMENDATION · RECOMMENDED DECISION
order writer	SUBJECT · DOCUMENT
update writer	HEADLINE · UPDATE

Each reply is parsed strictly; missing, extra or multi-line fields are refused.

The state that travels

`report_reference, coordinator_request, coordinator_id, thread_id, run_id, review_id, report_profile, report_documents, register_count, screening, evidence_findings, disagreements, likelihood, crew_hours, review_plan, requirements_analysis, evidence_review, prediction_analysis, review_decision, order_document, document_details, resident_update, llm_call_count`

`llm_call_count` is **added to** by every agent node (a reducer); every other key is replaced by the node that writes it. Each node returns only the keys it wrote.



Calls per run: 5 before the gate, then 0, 1 or 2 depending on the decision

AGENTS INTERPRET; TOOLS CALCULATE

Every figure an agent is shown was established by the standards, the register, the models, the documents and the tools. Each prompt tells its agent to use only what it is given. The review stores the five write-ups (each at least 8 words, no two the same) beside the facts they explain.

11. Flow 7: The human decision gate

The run stops before anything is decided or written. The pause lives in `database/checkpoint.db`, so it survives the service being switched off, and another process can resume it. Every check runs **before** the run resumes, so a refused decision costs no LLM call.

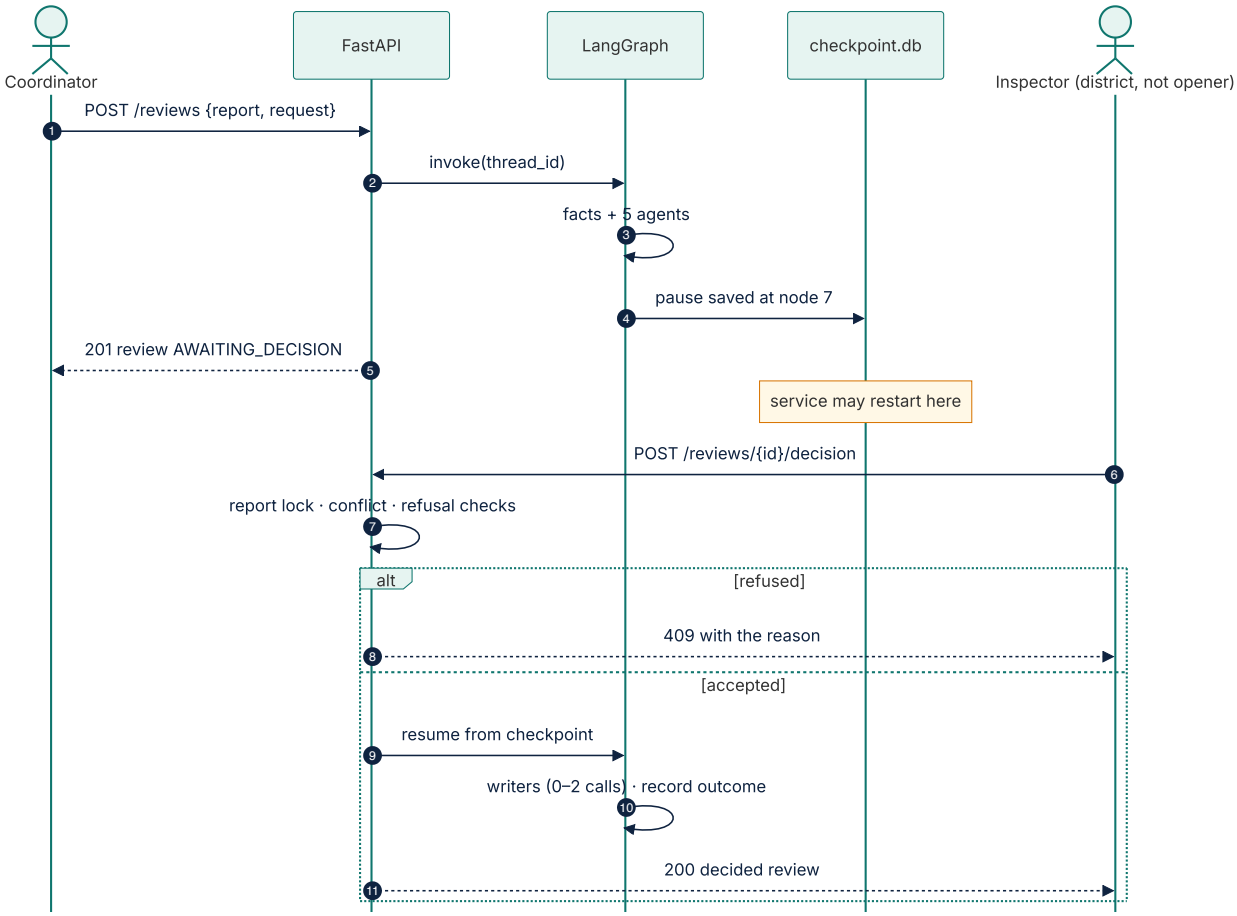


Figure 9: Open, pause, decide and resume

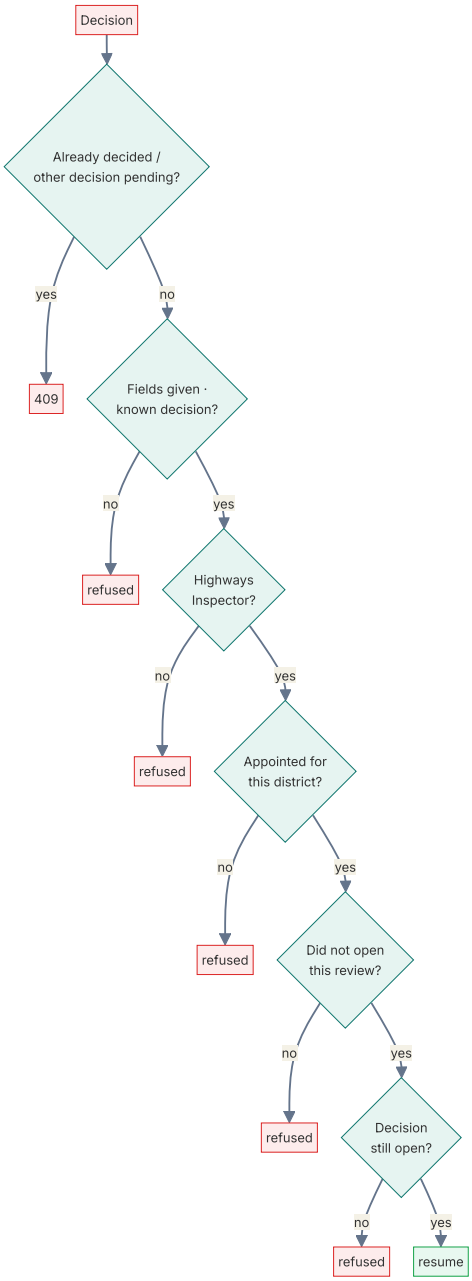


Figure 10: The first check that applies refuses the decision, in the configuration's own words

Decision	Writes	Total LLM calls	Review status
ORDER REPAIR	works order + resident update	7	REPAIR_ORDERED
INSPECT ON SITE	inspection request (records to take, disagreements to settle)	6	INSPECTION_REQUESTED
REFER FOR COORDINATION	coordination brief, built by code	5	REFERRED_FOR_COORDINATION

12. Flow 8: The secured REST service

Every figure the dashboard shows comes from this service. Sign-in and sign-out are public; every other route sits on one guarded router, so no endpoint can forget the check. The service refuses a request on its own account, whatever the dashboard shows.

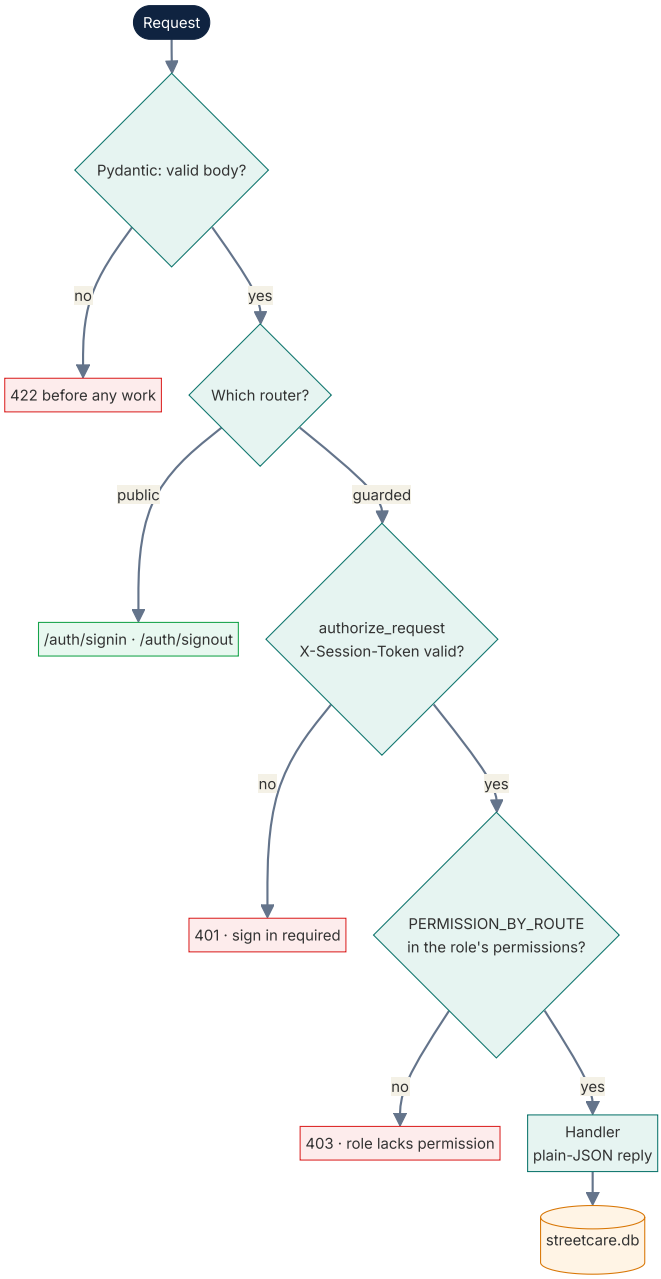


Figure 11: The path of every request

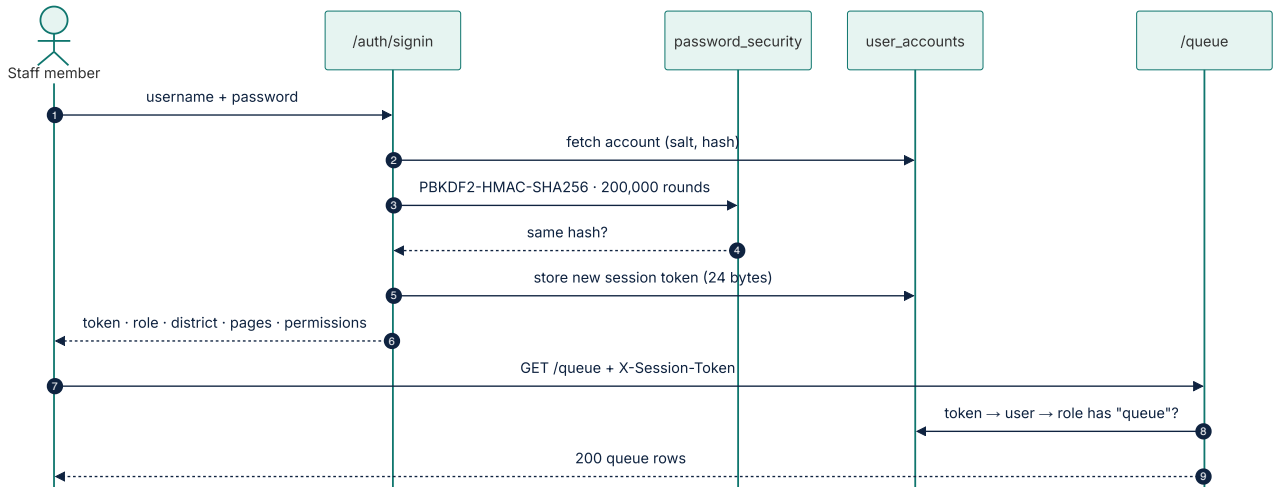


Figure 12: Sign in, then use the token

Method	Path	Permission	What it does
POST	`/auth/signin`	public	Check password, issue a session token, return pages and permissions
POST	`/auth/signout`	public	End the session
GET	`/queue`	queue	Every open report, Footway-linked first, then by reference
GET	`/queue/{report_reference}`	queue	Report, documents, screening, permits, evidence, flags, predictions
POST	`/reviews`	review	Run the workflow to the gate → **201** (costs 5 LLM calls)
GET	`/reviews`	audit	Review records, filter by status
POST	`/reviews/{id}/decision`	decision	Check, resume, write documents (costs 0–2 calls)
GET	`/reviews/{id}/calls`	audit	Every LLM call: tokens, latency, count vs expected
POST	`/chat/ask`	chat	Ask the street care assistant
GET	`/chat/turns`	chat	Your own recent chat turns
GET	`/audit/chat-turns`	audit	Everyone's chat turns, for audit

Permission	Highways Inspector	Street Care Coordinator	Customer Services Adviser
queue · review · audit	✔	✔	–
decision	✔ own district, not own review	–	–
chat	✔ full review records	✔ full review records	✔ status only, never a prediction

SERVICE START-UP

Before importing anything heavy, the service checks every required file (clean history, three models, two schemas, three live files, the documents folder and the index). If one is missing it prints *"The service cannot start."* and names each missing file with the step that makes it. Otherwise it loads everything once, works out the queue and every report's detail once, and gives each report its own lock.

13. Flow 9: The street care chat

Anyone signed in can ask questions. One Gemini call classifies the question into one of seven types and drafts an answer from the council's documents. **Code, never the model**, writes every answer about a report or a street, by the asker's role.

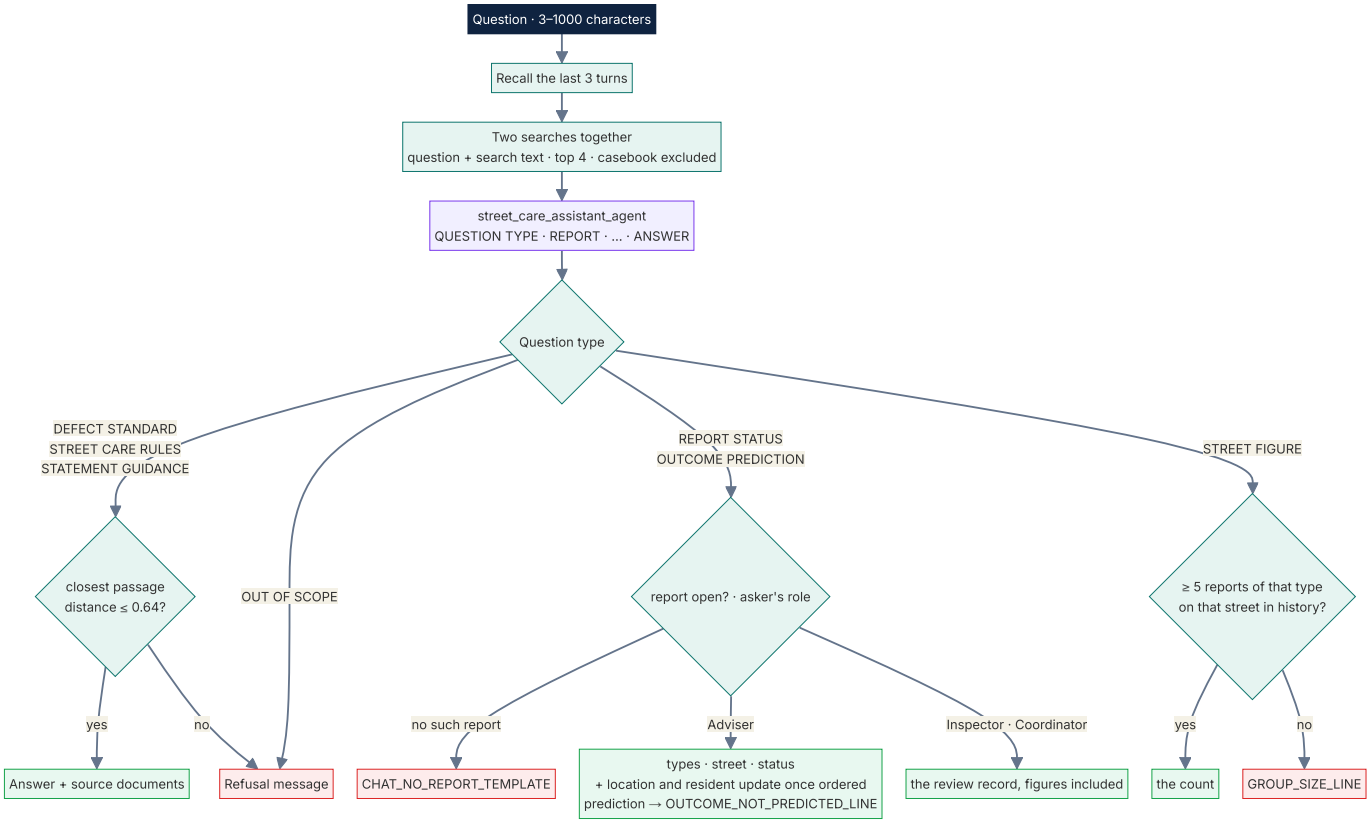


Figure 13: Seven question types; every turn is stored in chat_turns

TWO RULES THAT PROTECT RESIDENTS

- **No predictions to customer services.** Asked whether a defect will come back or how long a repair will take, an adviser hears one fixed line and nothing more.
- **No small groups.** A figure about a street is given only when at least 5 reports make it up, counted from the report history, never the live queue; below that it could point at the household that reported.

14. Flow 10: The dashboard

Streamlit on port 8080. The dashboard signs the user in through the service, then shows only the pages in `ROLE_PAGE_ACCESS`. It holds no business rule: every call goes through `api_client.call_service` with a 300-second timeout, a waiting message and a failure message.

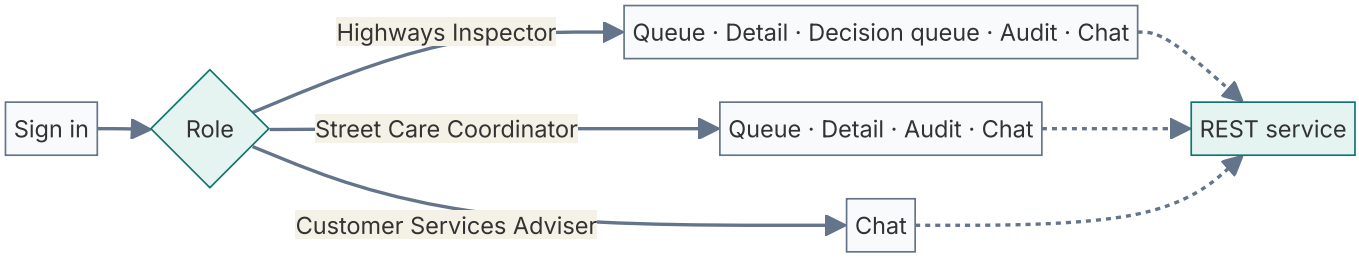


Figure 14: Each role sees only its pages

Page	Shows / does	Calls
Report queue	every open report, its status, counts, assessment, band, overrun, disagreements; filter by type	GET /queue
Report detail	answers, documents, each requirement's result, permits, triggers, evidence, flags; open a review	GET /queue/{ref} · POST /reviews
Decision queue	reviews awaiting a decision with the five write-ups; decide with a reason	GET /reviews · POST /reviews/{id}/decision
Audit trail	reviews, every LLM call with tokens and time, every chat turn	GET /reviews · /reviews/{id}/calls · /audit/chat-turns
Street care chat	ask questions; your last 20 turns	POST /chat/ask · GET /chat/turns

15. One review, end to end

A coordinator opens a review of one open report and an inspector from the same district orders the repair. Here is everything that happens.

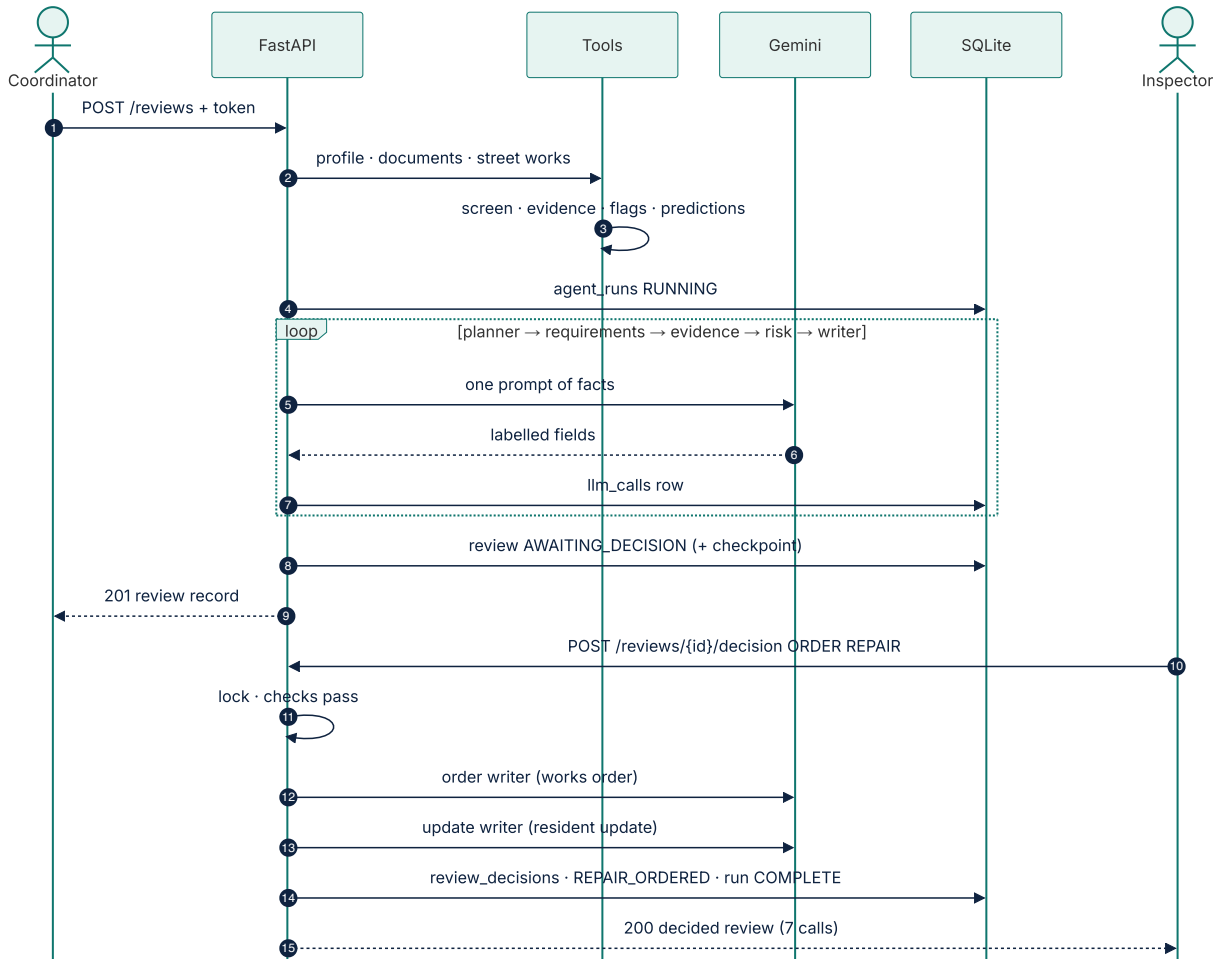


Figure 15: Full trace of one review

```
{ "review_id": "REV-3F9A1C0B7E2D", "run_id": "RUN-8C1D2E3F4A5B", "report_reference": "SC00xxxxx",
  "defect_type": "Carriageway", "street_name": "…", "district": "…",
  "requirements_met_count": 4, "requirements_not_met_count": 0, "requirements_not_documented_count": 0,
  "coordination_triggers": [], "assessment": "ORDER REPAIR", "decisions_open": ["ORDER REPAIR",
"INSPECT ON SITE", "REFER FOR COORDINATION"],
  "repeat_likelihood": 0.xx, "likelihood_band": "…", "expected_crew_hours": x.x, "crew_hours_text":
"…",
  "recommendation_text": "…", "recommended_decision": "ORDER REPAIR",
  "status": "REPAIR_ORDERED", "order_document_text": "… Street Care Office, Wrenford Borough Council",
  "resident_update_text": "…", "llm_call_count": 7 }
```

Illustrative shape of the stored review (field names are the specification's). Every figure in it comes from code or a model, never from the LLM; the outgoing documents carry no model figure at all.

16. Files: who writes, who reads

File	Written by	Read by
`database/streetcare.db`	run_database_setup; then the service	sign-in, workflow nodes, routes, chat, audit pages
`processed/report_history_*_clean.csv`	run_data_pipeline	model training; chat street figures
`processed/repair_logs_*_clean.csv`	run_data_pipeline	crew hours training
`processed/statement_sentences_clean.csv`	run_data_pipeline	sentence training
`output/data_quality_report.json`	run_data_pipeline	reviewers: proof each repair was needed
`output/models/repeat_model.pkl` + `repeat_features.json`	run_model_training	prediction tools (likelihood)
`output/models/crew_hours_model.pkl` + `crew_hours_features.json`	run_model_training	prediction tools (crew hours)
`output/models/sentence_model.pkl`	run_model_training	evidence tools
`output/model_metrics.json`	run_model_training	reviewers: held-out scores and gaps
`database/chroma_store/`	run_knowledge_indexing	retrieval tools, standard sections, chat search
`database/checkpoint.db`	the workflow's checkpointer	resuming paused reviews after a restart
`data/input/*.csv` (read-only)	the council's channels	the service at start-up (queue, documents, register)

The six audit tables

Table	One row is...	Key columns
user_accounts	a person who can sign in	username · role · district · password_hash · password_salt · session_token
agent_runs	one review run	run_id · thread_id · report_reference · coordinator_id · status · llm_call_count
report_reviews	the stored review (37 columns)	facts, requirements, permits, triggers, assessment, evidence, figures, 5 write-ups, status, documents
review_decisions	who decided what and why	decision · decided_by · role · district · decision_reason
llm_calls	one accepted LLM call	agent_name · model_name · prompt/response/total tokens · latency_milliseconds
chat_turns	one chat question and answer	user · type · report · street · answer · sources · closest_distance · was_refused

17. Design choices worth explaining

Rules live in the documents

No requirement or figure is in code or config. Each is read from the report's own standard at run time, so the near-identical carriageway and footway standards can never be confused.

One source of truth for settings

Every path, mapping, pattern, threshold, role and message lives in `config/config.py`, imported and never retyped.

Refuse, never guess

An unknown spelling, unit or identifier stops the run and names the value. A wrong answer that looks right is worse than a clear error.

Exact decimals, rounded once

`Decimal` and half-up rounding at each reading's own precision, so limits are met or missed exactly as the Report Rules say.

Tools calculate, agents interpret

Every number comes from code or a model. Each agent is one LLM call that explains facts it is given, so its output can be checked against them.

A person decides

The workflow pauses before any decision. Only a Highways Inspector appointed for the district, who did not open the review, may decide.

Pause survives restarts

LangGraph checkpoints to SQLite through a connection usable from more than one thread, so another process can resume a paused review.

Checks before cost

Conflict and refusal checks run under a per-report lock before the run resumes, so a refused decision spends no LLM call and two decisions can't race.

Predictions inform only

They never order the queue, set the assessment or appear in outgoing documents, and customer services never hears them.

Privacy floor of five

No figure about a street from fewer than five reports, so no answer points at one household.

Preparation inside the pipeline

Imputers, scalers, encoders and TF-IDF are fitted on training data only and saved with each model, so live scoring matches training and nothing leaks.

Audit everything

Every run, review, decision, accepted LLM call (tokens, latency) and chat turn is stored, and calls are counted against the 5 / 6 / 7 expected.

One guarded router

All protected routes hang off a router guarded by `authorize_request`, so no endpoint can forget the permission check.

Thin dashboard

Streamlit holds no rule; hiding a page is a convenience, while the API enforces access.

18. How to run it

REMEMBER

The first indexing run downloads the embedding model (`all-MiniLM-L6-v2` , about 90 MB). Everything except the LLM-backed steps (reviews and chat) works without a Gemini key. Every command is run from the `Project/` folder.

```
# 0. set up
cd Project
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
# put GEMINI_API_KEY and GEMINI_MODEL in .env (never commit it)

# 1-4. build, once and in order
python3 run_database_setup.py        # 6 tables · 16 accounts (safe to repeat)
python3 run_data_pipeline.py         # 5 clean CSVs · data_quality_report.json
python3 run_model_training.py        # 3 models · model_metrics.json (stops if any is below its
minimum)
python3 run_knowledge_indexing.py    # 16 documents · 136 sections · chroma_store

# 5. the service (terminal 2, keep it running) → http://localhost:8081/docs
python3 run_api_service.py

# 6. the dashboard (terminal 3) → http://localhost:8080
python3 -m streamlit run run_dashboard.py

# tests
python3 -m pytest tests.py -v
```

Situation	What to do
API says "The service cannot start"	run the step named beside each missing file
Machine restarted while a review waits	start the API again; the review resumes from `checkpoint.db` when decided
A defect standard PDF changed	rerun indexing, then training (requirement counts change), then restart the API
New live reports	replace the files in `data/input/` and restart the API
A decision failed half-way	send the same decision again; it finishes the run
Rebuild from clean	delete `processed/` , `output/` , `database/` and rerun steps 1–4

BEFORE YOU SHARE THE REPOSITORY

`Project/.env` holds a real Gemini key and is committed to git, and `.github/workflows/build.yml` has a Sonar token written into it. Rotate both, move the token to repository secrets and add `.env` to `.gitignore` .